

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Deepa preya H	Reg.No.:	192421047
Department:	Information technology	Date:	28.08.26

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
 - Maximum interrupt latency < 10 μ s
 - Use vectored interrupts for high-priority devices
 - Use software interrupts for background tasks
-

Code of Conduct:

I Deepa preya H / 192421047 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Real-Time Embedded System – I/O Communication Mechanism

Problem Understanding

The system must handle multiple sensors generating interrupts every **1 ms**, while keeping CPU utilization **below 15%**. Vectored interrupts are required, and the I/O technique must provide low latency.

Proposed Solution

Use **memory-mapped I/O with vectored interrupt handling**.

Architecture:

Sensors → Interrupt Controller → CPU → Memory-Mapped I/O Registers

1. Each sensor is connected to an interrupt line.
2. An **interrupt controller** collects sensor interrupts.
3. **Vectored interrupt handling** directly identifies the ISR associated with each sensor.
4. Sensor registers are mapped into the CPU's memory address space.
5. The ISR reads the sensor data and stores it in a buffer.
6. A **ring buffer** can be used to prevent data loss when interrupts occur frequently.

Why Memory-Mapped I/O?

Feature	Memory-Mapped I/O	I/O-Mapped I/O
Address space	Shared with memory	Separate I/O address space
Instructions	Normal load/store	Special I/O instructions
Programming	Easier	More specialized
Latency	Low	Can be slightly higher
Choice	Selected	Not selected

Conclusion: Memory-mapped I/O with vectored interrupts provides fast access and reduces ISR overhead, helping maintain CPU utilization below 15%.

2. High-Speed Storage I/O Subsystem

The system requires **more than 3 GB/s throughput**, must use **NVMe over PCIe**, and the CPU should not perform bulk data transfers. DMA with interrupt notification is required.

Proposed Architecture

NVMe SSD → PCIe → DMA Controller → Main Memory

↓

Interrupt → CPU

Working

1. The NVMe SSD communicates through **PCIe**.
2. The CPU initializes the NVMe command and DMA descriptors.
3. The **DMA controller** transfers blocks directly between NVMe and main memory.
4. The CPU does **not copy the bulk data**.
5. After completing the transfer, DMA generates an **interrupt**.
6. The CPU processes the completion notification.

Advantages

- High data-transfer speed.
- CPU overhead is minimized.
- Suitable for throughput above **3 GB/s**.

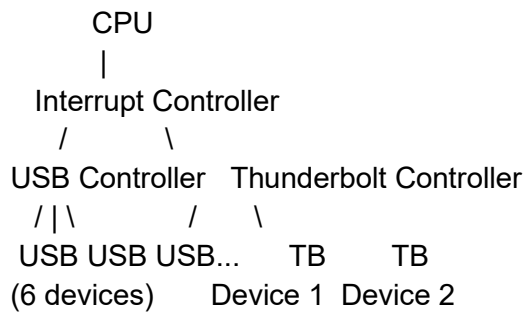
- DMA enables direct memory transfer.
- Interrupts notify the CPU only after important operations.

Conclusion: NVMe over PCIe combined with DMA provides high throughput while minimizing CPU intervention.

3. Multi-Device Communication Architecture

The system contains **6 USB peripherals and 2 high-speed Thunderbolt devices**. It must avoid interrupt starvation, support hardware/software interrupts, and provide fair bus arbitration.

Architecture



Working

1. The **USB controller** manages the six USB peripherals.
2. The **Thunderbolt controller** manages the two high-speed devices.
3. Hardware interrupts are generated by devices requiring immediate CPU attention.
4. Software interrupts are used for lower-priority/background processing.
5. An interrupt controller assigns priorities to avoid **interrupt starvation**.
6. **Fair bus arbitration** can use a round-robin approach so that each device gets bus access.

Interrupt Priority

High priority → Time-critical devices

Medium priority → USB/Thunderbolt transfers

Low priority → Background software tasks

Why Round-Robin Arbitration?

Round-robin prevents one high-traffic device from continuously occupying the bus. Each device receives an opportunity to transmit.

Conclusion: Separate controllers with priority-based interrupt handling and round-robin bus arbitration provide fair and efficient communication.

4. Data Acquisition System

The system continuously produces **500 MB/s**, requires minimal CPU intervention, must compare DMA with interrupt-driven I/O, and requires memory-mapped I/O for device registers.

Proposed Architecture

Data Acquisition Device → DMA Controller → Main Memory

↓

Interrupt → CPU

Device control/status registers are accessed using **memory-mapped I/O**.

DMA vs Interrupt-Driven I/O

Feature	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Suitable for 500 MB/s	Yes	No
Data transfer	Directly to memory	CPU handles transfer
CPU overhead	Low	High
Performance	High	Lower
Selected	Yes	No

Working

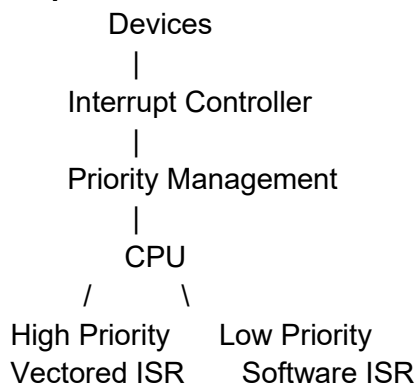
1. The CPU configures the data acquisition device.
2. Device registers are accessed using memory-mapped I/O.
3. DMA is configured with the destination memory address.
4. The device continuously generates data at **500 MB/s**.
5. DMA transfers the data directly to memory.
6. DMA generates an interrupt after a block transfer is completed.
7. CPU processes the completed buffer.

Conclusion: DMA is preferred because transferring 500 MB/s using interrupts would cause excessive CPU overhead.

5. Interrupt Handling Mechanism

The system must support **nested interrupts**, maintain maximum latency below **10 µs**, use vectored interrupts for high-priority devices, and use software interrupts for background tasks.

Proposed Architecture



Working

1. The interrupt controller receives interrupts from different devices.
2. Each interrupt is assigned a priority.
3. High-priority interrupts use **vectored interrupt handling**.
4. The CPU immediately jumps to the corresponding ISR.
5. **Nested interrupts** allow a higher-priority interrupt to interrupt a lower-priority ISR.
6. After servicing the higher-priority interrupt, the CPU returns to the interrupted ISR.
7. Background operations use **software interrupts**.
8. Short ISRs and priority-based handling help maintain latency below **10 µs**.

Example

Suppose:

- Network interrupt = High priority
- Sensor interrupt = Medium priority
- Background task = Low priority

If the CPU is processing the sensor ISR and a network interrupt occurs:

Sensor ISR → Network ISR → Return to Sensor ISR → Continue

Conclusion

Vectored interrupts, priority-based nesting, and short ISRs provide fast response for critical devices while software interrupts handle background operations efficiently.